pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

One-Dimensional Convolutions and Many Tenors

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Three ways to read a window

Dense, day 11

Every day its own weight: the window laid flat.

Recurrent, day 12

Day by day, a state carried forward.

Convolution, today

Five weights slid along the window.

Past tense: the 10 Yr change in bp on a date, from the 20 trading days before it.

Allowed names

fitted change in bp

walk-forward error

the last value carried forward

Never called

a forecast or a prediction

tomorrow's move or a call

an outlook or a view on rates

How a convolution works

1. A filter

a few weights laid against consecutive days

2. One output

multiply day by day, add, add the bias

3. Slide

one day along; the same weights again

4. The result

a new sequence per filter

You control

The width, the number of filters, the padding, and the stride.

What comes out

One number per position, for each filter.

A filter sliding over time



20 daily changes of the 10 Yr before 2015-02-03; a filter of width 5 covers days 1 to 5, then 2 to 6, ...

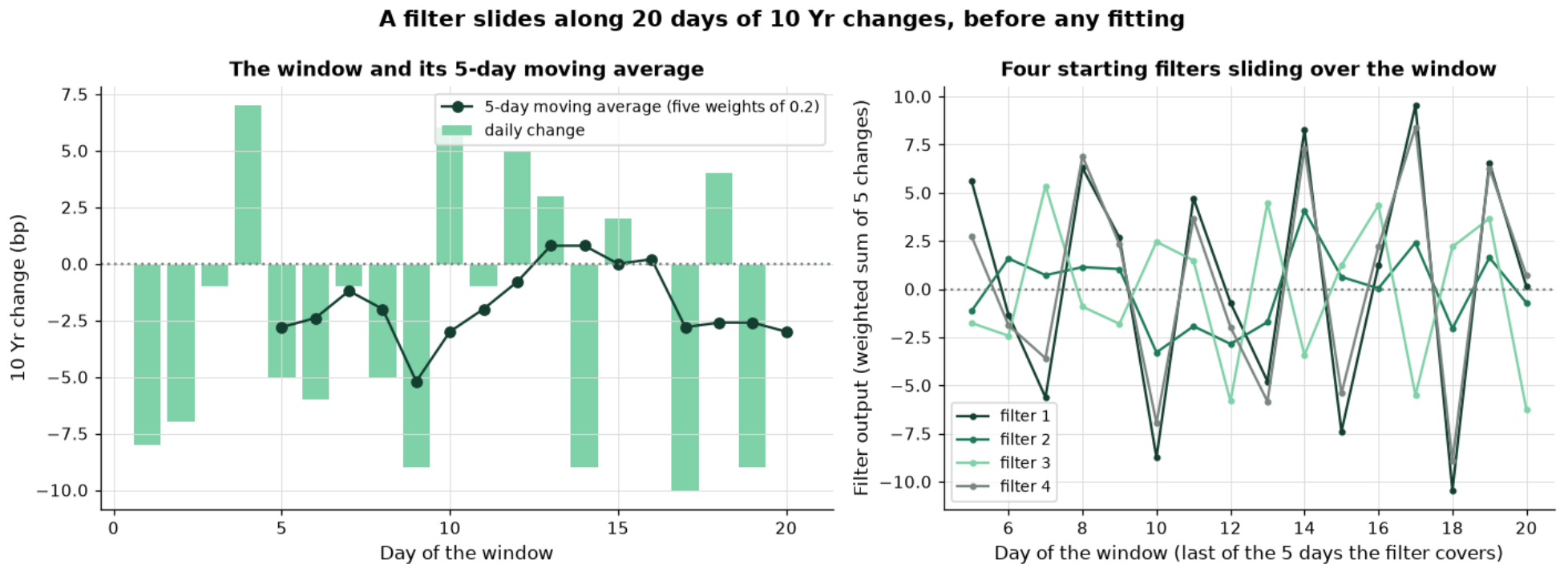
```
output[t] = sum of window[t + k] * weight[k] for k = 0 to 4, plus the bias
outputs: 20 - 5 + 1 = 16          kernel shape (width, channels, filters): (5, 1, 4)
```

Five weights of 0.2 are a 5-day moving average

First output -2.8 bp; equal to pandas' rolling(5).mean() within 1e-12.

A network fits its own weights: each filter is a moving sum with weights of its choosing.

The window and its moving average



A filter of equal weights smooths; four random filters give four different weighted moving sums. Nothing is fitted yet.

A filter is a weighted moving sum

	Day 1	Day 2	Day 3	Day 4	Day 5
Change, bp	-8.0	-7.0	-1.0	7.0	-5.0
Weight	0.351	-0.4134	0.0233	0.4566	-0.4714

Products added, plus a bias of 0: 5.6165

Same five weights at every step

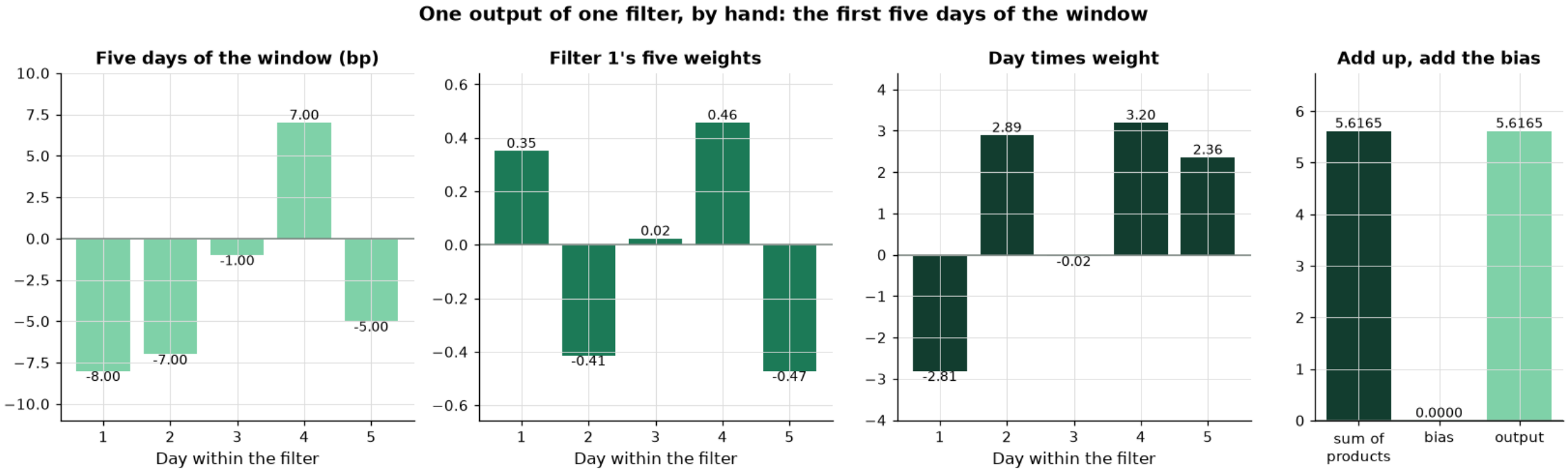
Day 11's dense network gave each day its own weight; a filter has 5 however long the window.

11 tenors as channels: kernel (5, 11, 4)

A filter is a block of 5 days by 11 tenors, 55 weights. Each output is one SUMPRODUCT over a rectangle; the output is still 16 steps per filter.

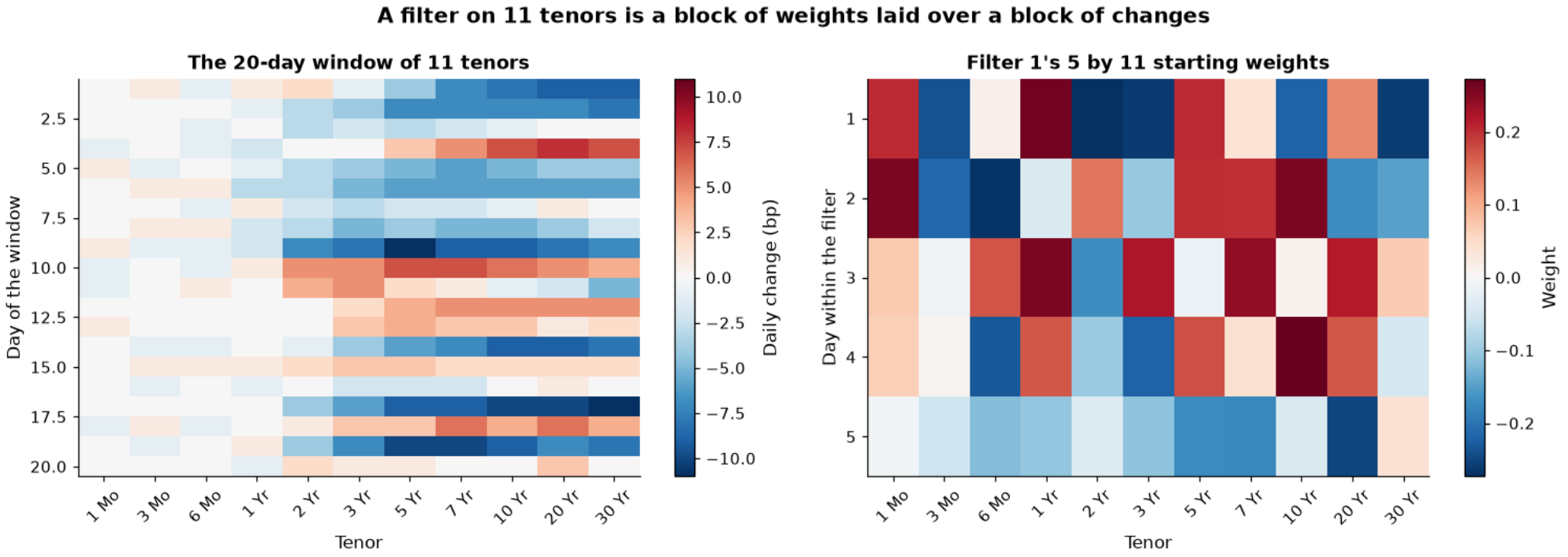
Keras computes a cross-correlation: the kernel is not flipped.

One output of one filter, by hand



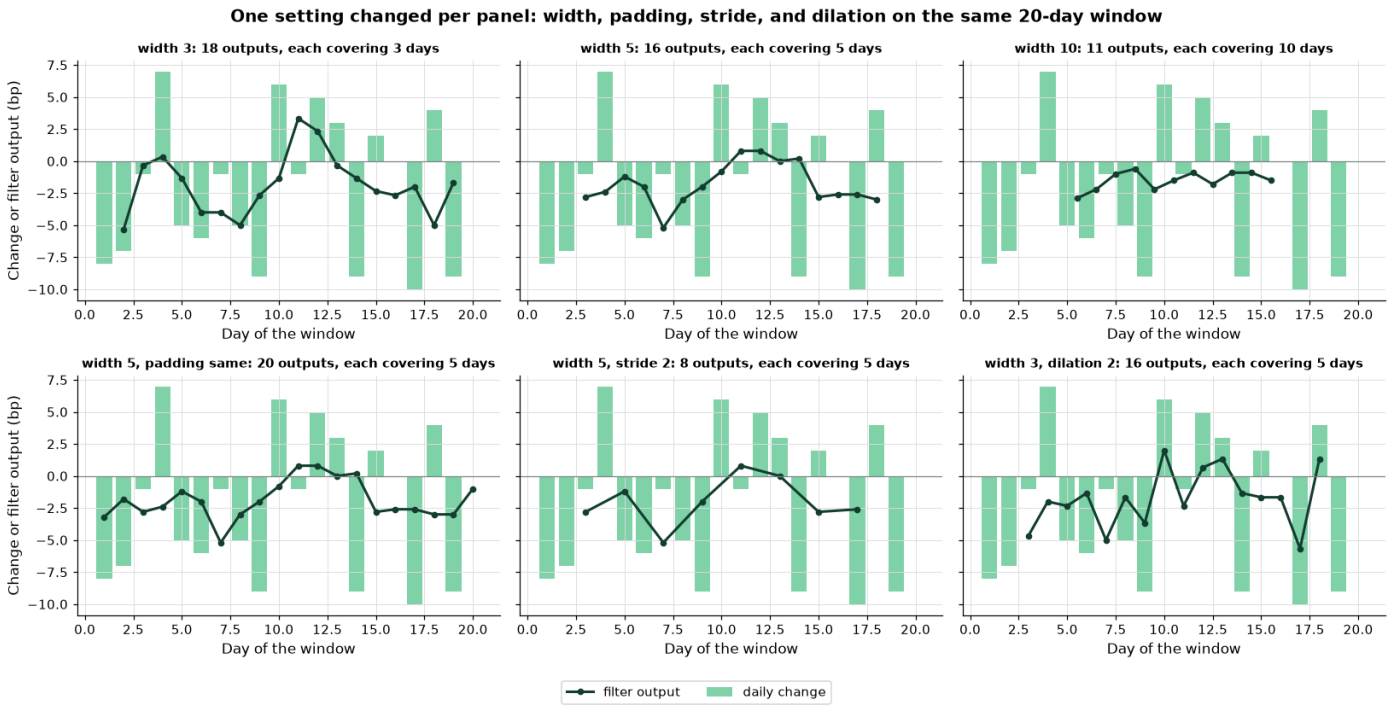
The products add up to the first value conv1d_by_hand returned; every output is this picture on the next five days.

A block of weights on a block of changes



Each output multiplies a 5-day slice of the left grid, cell by cell, by the right grid and adds up: one SUMPRODUCT over a rectangle.

Width, padding, stride, and dilation



One setting changed per panel. The course uses valid padding, stride 1, and no dilation.

Every setting of Conv1D

Setting	Default	What it controls
filters	required	how many patterns; tune first
kernel_size	required	the filter's width in days; tune first
strides	1	days between outputs
padding	'valid'	valid, or same with zeros at the ends
dilation_rate	1	the gap between the days one filter looks at
activation	None	applied to each output; tune first

16 settings in all; the notebook's table lists every one, with what raising and lowering it does.

How pooling works

1. Global

one number per filter over every position

2. Average

how much the pattern showed up

3. Maximum

where it showed up most

4. Windowed

one number per block; a shorter sequence

You control

Which summary; for windowed pooling, the block size and the step.

What comes out

One number per filter, or a shorter sequence.

Pooling: one number per filter

7 of 16

filter 1's outputs set to 0 by ReLU

2.8086

average pooling, filter 1

9.5347

max pooling, filter 1

```
keras.layers.Conv1D(16, 5, activation="relu")   # (20, 1) -> (16, 16)
keras.layers.GlobalAveragePooling1D()           # (16, 16) -> (16,)
keras.layers.Dense(1)                            # (16,) -> 1, a change in either sign
```

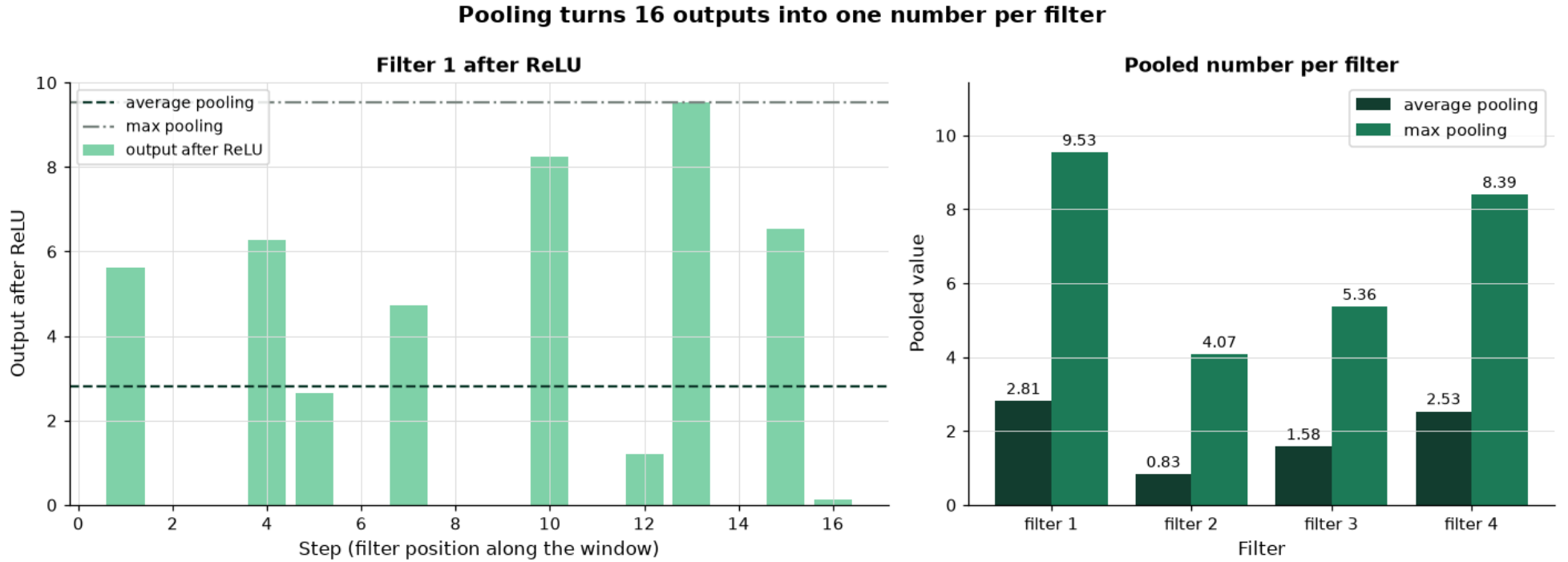
113

weights and biases, one tenor

913

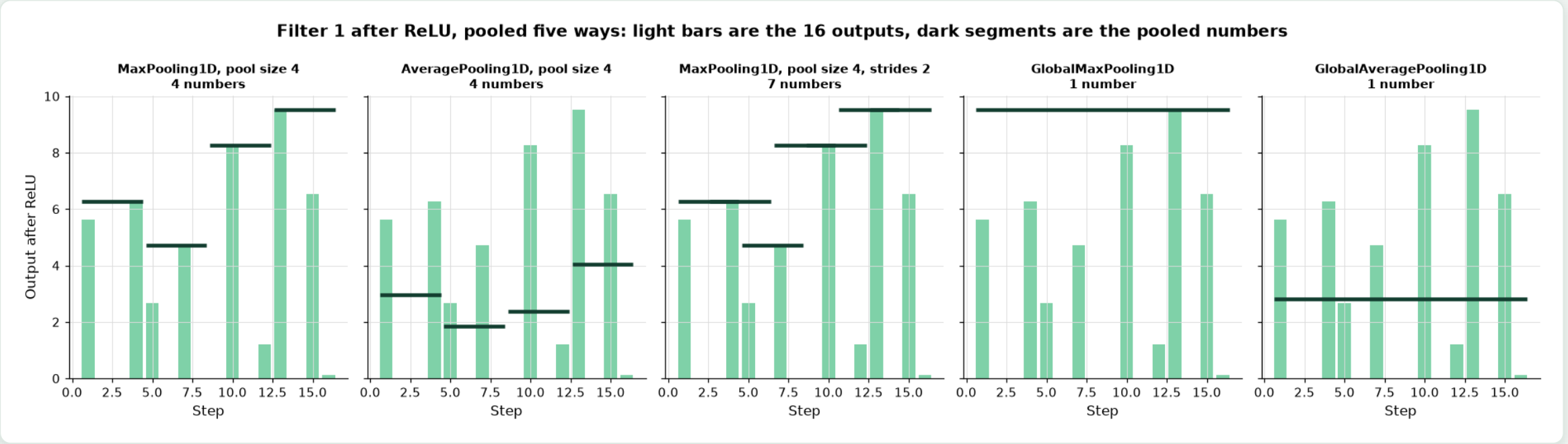
weights and biases, 11 tenors

Pooling, drawn



ReLU set 7 of filter 1's 16 outputs to 0; pooling turns what is left into one number per filter.

Pooling, five ways



Windowed pooling keeps roughly where a pattern showed up; global pooling keeps how strongly. The course uses the global average.

Every setting of the pooling layers

Setting	Max	Average	Global max	Global avg
pool_size	2	required	-	-
strides	None	None	-	-
padding	'valid'	'valid'	-	-
data_format	None	None	None	None
keepdims	-	-	False	False

A dash means the layer has no such setting. The course's network uses GlobalAveragePooling1D with every setting at its default.

Walk-forward: the Conv1D on the 10 Yr

Model, 10 Yr alone	MAE, bp	Folds below zero change
zero change	3.21	-
training mean	3.22	-
dense, day 11	3.35	0 of 5
SimpleRNN, day 12	3.31	0 of 5
LSTM, day 12	3.23	0 of 5
GRU, day 12	3.22	1 of 5
Conv1D, today	3.24	0 of 5

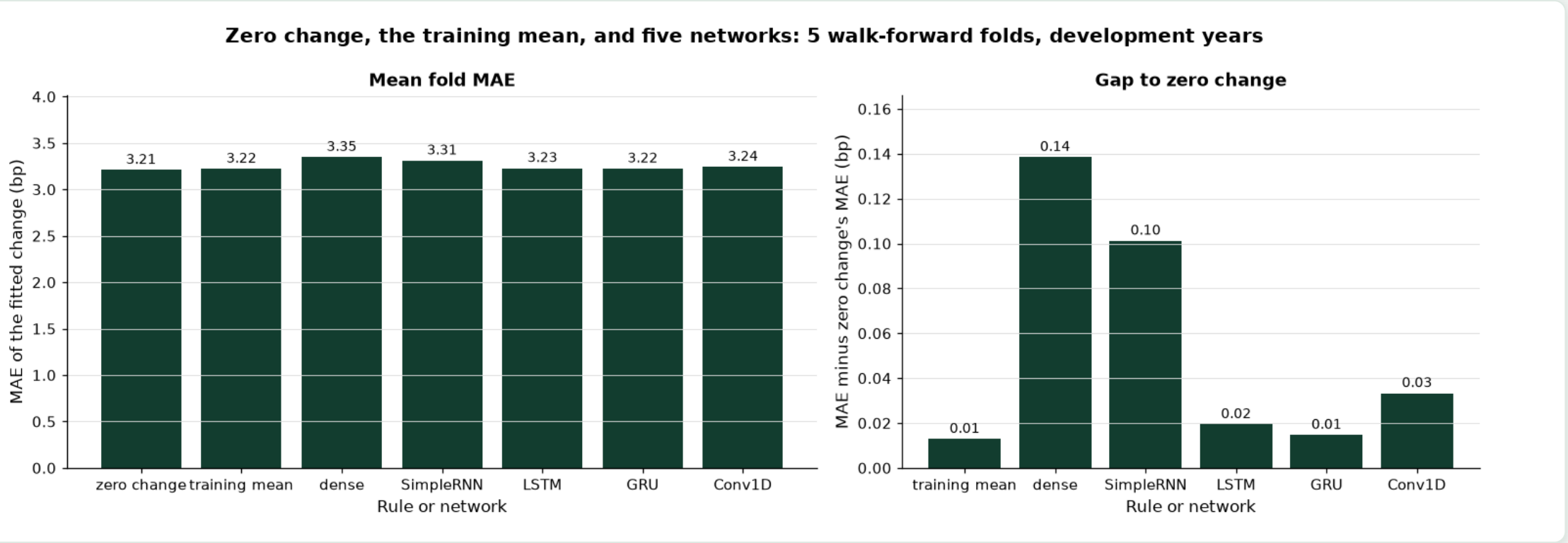
Best epoch by fold, Conv1D

```
1, 50, 1, 1, 50
```

Fitted changes spread 0.25 bp against the actual 4.33 bp: little to fit, so it stays near the training mean.

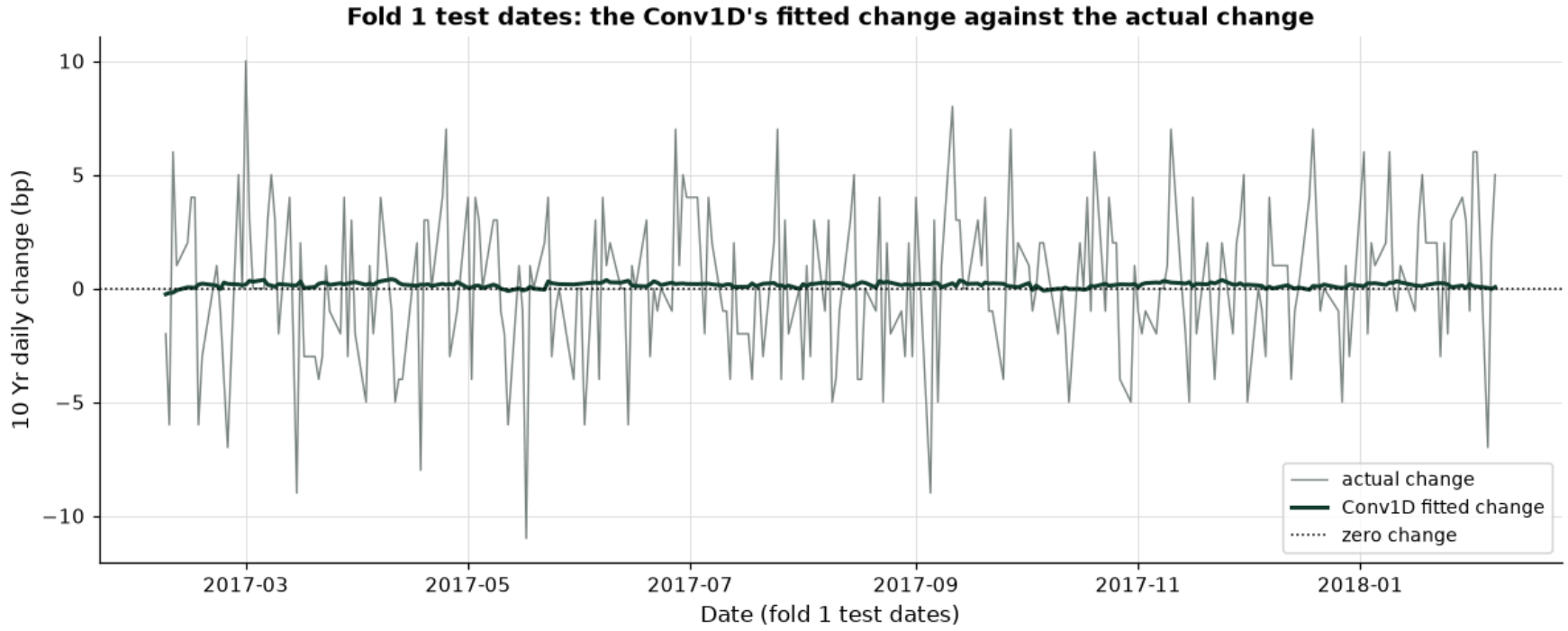
Same five folds, same settings, reseeded every fold: days 11 and 12 rebuilt here.

Seven scores as bars



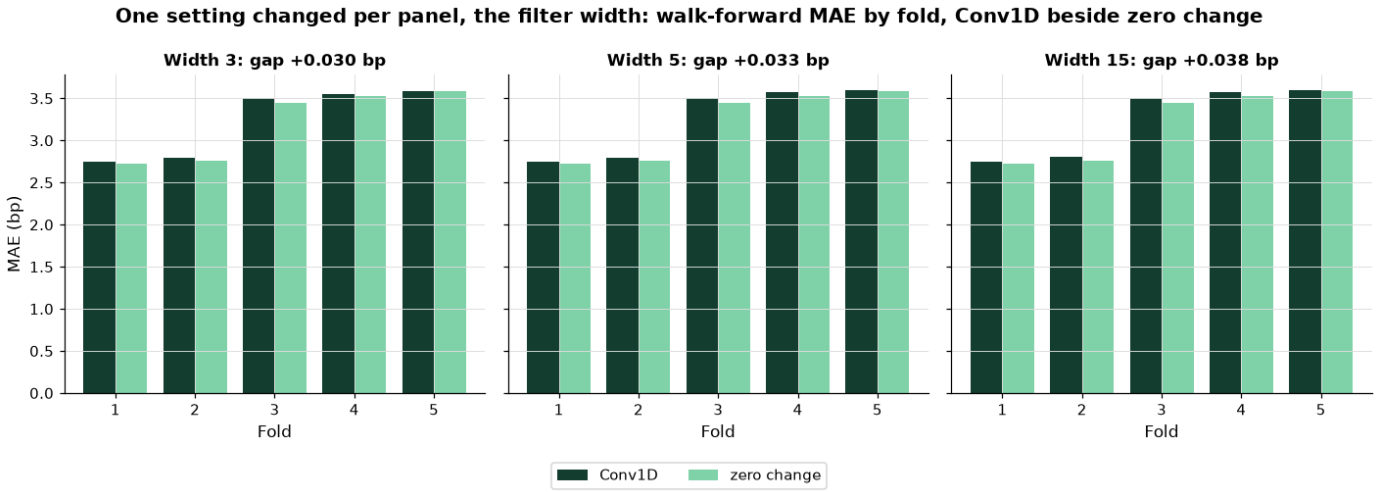
Zero change, 3.21 bp, is the lowest bar; every other bar sits above it.

The Conv1D on fold one's dates



The fitted changes spread 0.25 bp against the actual 4.33 bp: a network that finds little to fit stays near the training mean.

The filter width, one setting changed per panel



Width	Weights	Network	Zero change	Gap	Folds below
3	81	3.239	3.209	0.030	0
5	113	3.242	3.209	0.033	0
15	273	3.247	3.209	0.038	0

Many tenors as channels

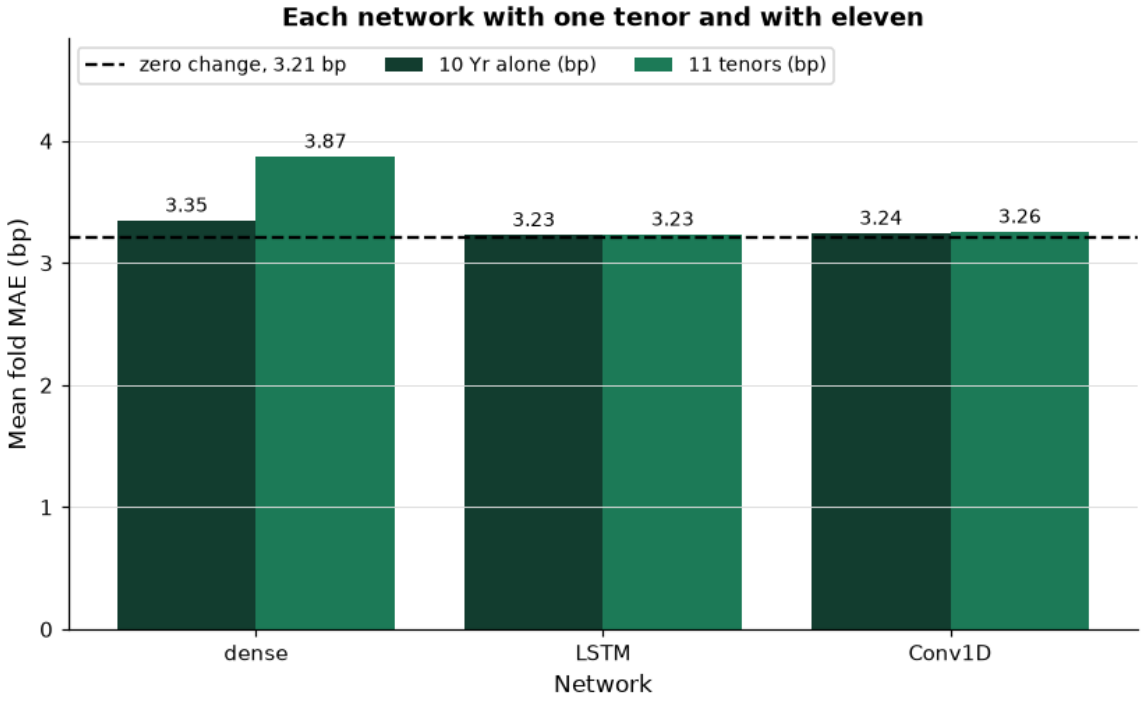
Network	10 Yr alone, bp	11 tenors, bp	Folds below zero change
dense	3.35	3.87	0 of 5
LSTM	3.23	3.23	1 of 5
Conv1D	3.24	3.26	0 of 5

Zero change on the same folds: 3.21 bp.

The 11-tenor Conv1D is not bit-identical run to run in its last digits (its MAE agreed to 6 decimals): printed as an MAE to 2 decimals and fold counts only.

The flattened dense network got worse; the shared-weight networks barely moved.

One tenor or eleven, as bars



The dense network's pair moves; the LSTM's and the Conv1D's are nearly equal. Every bar is above zero change.

Same-day changes stay out

10 Yr change against	Same date	One date earlier
2 Yr change	0.75	0.02
30 Yr change	0.93	-0.03

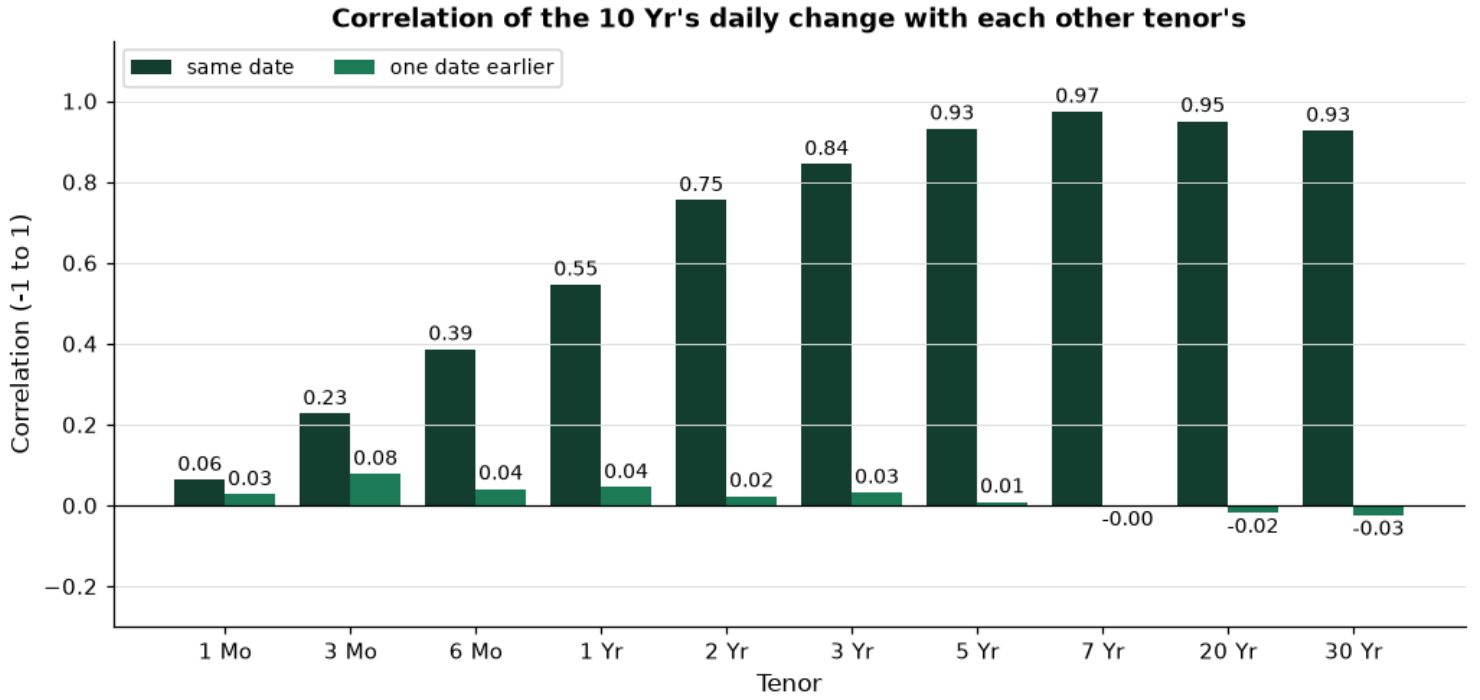
Known only after the day

A window that ends on its target date holds other tenors' same-day changes.

```
netkit.assert_windows_past_only(changes.index[positions - 1], target_dates)   # passes
netkit.assert_windows_past_only(changes.index[positions], target_dates)
# AssertionError: 1980 of 1980 windows end on or after the date they are scored on
```

make_windows: all 1,980 windows end the trading day before their target date.

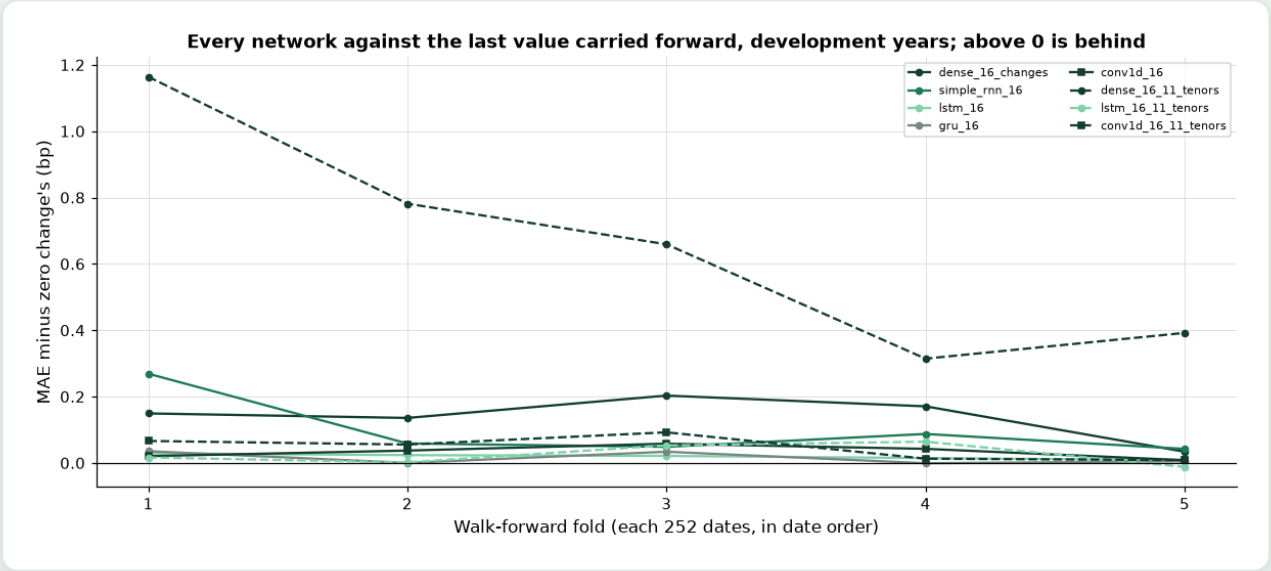
Correlation, tenor by tenor



Same-date bars rise from the shortest tenors to the longest; one-date-earlier bars sit near zero. One row too many in a window is the difference.

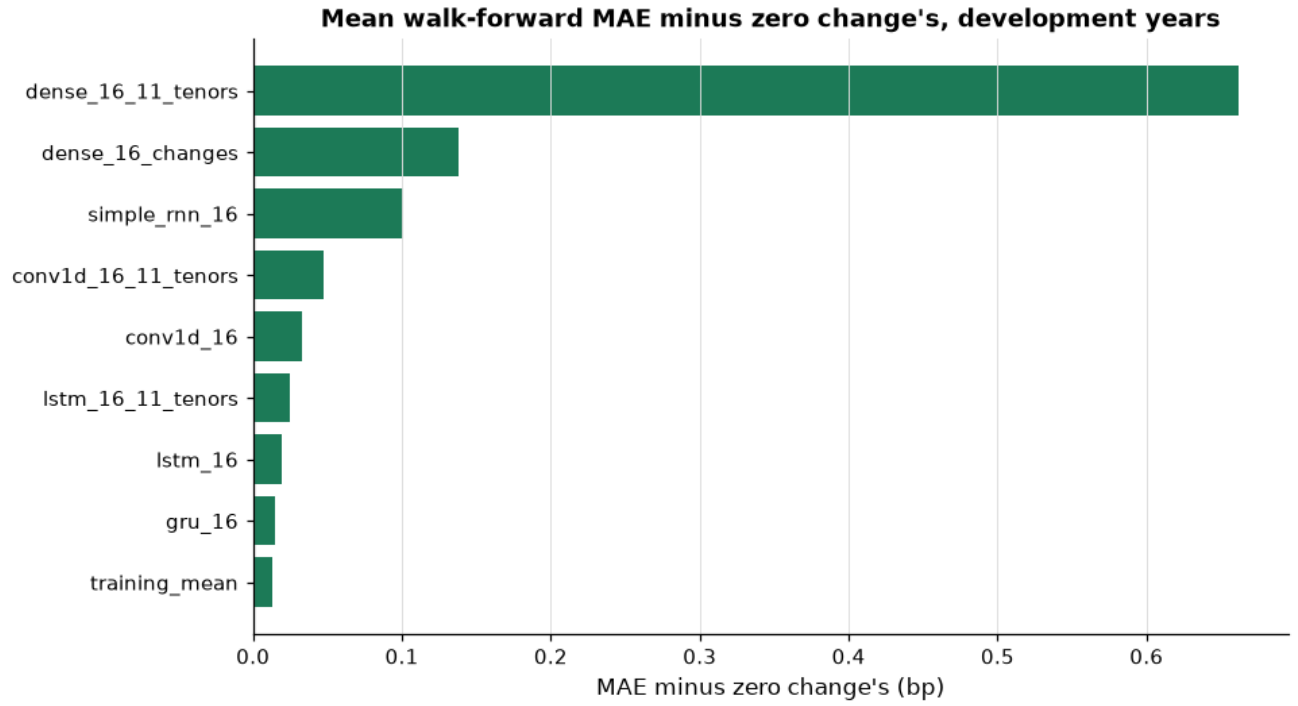
Every sequence model so far

Model	Ch.	MAE, bp	Below
zero_change	1	3.21	0
training_mean	1	3.22	0
dense_16_changes	1	3.35	0
simple_rnn_16	1	3.31	0
lstm_16	1	3.23	0
gru_16	1	3.22	1
conv1d_16	1	3.24	0
dense_16_11_tenors	11	3.87	0
lstm_16_11_tenors	11	3.23	1
conv1d_16_11_tenors	11	3.26	0



Zero change is the lowest line; no network beat it on average.

Gap to zero change, longest first



The dense networks are the long bars; the LSTMs, the GRU, and the convolutions are short and close together. Every bar is above zero.

The candidates for day 14

3.22 bp

gru_16, 10 Yr alone: lowest mean MAE

3.23 bp

lstm_16, 10 Yr alone: second lowest

3.21 bp

zero change, beside them on day 14

The rule

The two lowest mean walk-forward MAEs over the five development folds, on either input; every 11-tenor network was above its 10 Yr version. Chosen on development dates only; no held-out date fitted or scored.

Chosen is not good: neither beats zero change, and the seed spread is day 14's.

Excel side by side

Excel side by side (tested in Excel)

`=SUMPRODUCT(B2:B6,$K$2:$K$6)+$K$7`	C2 to C17: filter 1, first 5.6165
`=SUMPRODUCT(B2:B6,$M$2:$M$6) = =AVERAGE(B2:B6)`	five weights of 0.2: a moving average
`=MAX(0,C2)`	ReLU, filled down
`=AVERAGE(D2:D17) and =MAX(D2:D17)`	pooling: 2.8086 and 9.5347
`=SUMPRODUCT(B2:L6,$N$2:$X$6)+$N$7`	11 tenors: a 5 by 11 rectangle

Equal to netkit within 1e-12 and to Keras within 1e-5. Excel cannot fit the weights, and has no walk-forward routine.

Save the day: the sequence log

1. Morning

Files as day 12 left them

2. Code

conv1d_by_hand, assert_windows_past_only

3. Log, rebuilt

Days 11 and 12, no levels row

4. Today's rows

Conv1D, and 11-tenor networks

```
!git -C "{work}" log --oneline --decorate
# (HEAD -> branch) marks the newest commit; day 14 adds a tag
```

Columns: model, channels, window, rows, folds, mae_bp, zero_change_mae_bp, folds_below_zero_change. 56 tests pass.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: 11-tenor windows end on the target date: fold 1 2.09 bp against zero change's 2.72 bp.

Code that runs is not the same as code that is right.

What you can do now

- Compute a convolution by hand; check it against Keras
- Write a filter in Excel as SUMPRODUCT filled down
- Pool a filter's outputs: average and max
- Score a Conv1D walk-forward against zero change
- Keep same-day changes out of every window

Week 3, Day 4 Case study: the held-out years, opened once